obfusimg 简要文档

___roselle___, pigeon.hannah

February 21, 2026

Abstract

obfusimg 应用提供了多种算法，用于以确定且可逆的方式打乱图像像素，从而实现图像混淆。本文档作为该应用 C++ 实现的一份半技术型使用说明。

1 引言

可逆图像混淆有着广泛的应用场景，从保护隐私的图像分享（例如绕过审查）到科学与工业图像流程中的可逆预处理（例如在空间相关性被打乱的条件下对机器学习架构进行基准测试）。

一种相对直接的混淆方法是重新排列给定图像中的全部像素。每个像素按照某种规则被移动到一个唯一的新位置，使得输出图像具有相同的尺寸（宽度、高度、通道数），但几乎无法辨认（见图 1）。

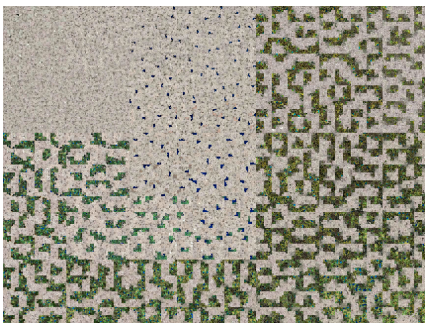
应用 obfusimg 实现了多种生成此类像素重排的方法。尤其是，这些算法的计算成本较低，用户可以在任何合理的本地设备上运行该应用，这意味着在混淆之前无需通过互联网传输任何信息。

2 图像与混淆的数学模型

设一幅图像的宽为 w px、高为 h px，并具有 c 个通道（通常 jpeg 文件的 RGB 图像有 $c = 3$，png 文件的 RGB+Alpha 图像有 $c = 4$）。我们将图像识别为一个 1 维数组

$$A = \left(a_0, a_1 \ldots, a_{wh-1}\right) \tag{1}$$

(a) 一张孔雀图像（由 mnrva 提供）。



(b) 所有像素被重排后的孔雀图像。

Figure 1: 示例图像中的孔雀在所有像素被重排后变得无法辨认。当前最先进的 LLM 也无法稳定识别该混淆图像的主体（在 ChatGPT 5.2 Thinking extended 上测试）。

其中每个 a_i 存储一个像素的全部 c 个通道值（通常每个通道值是整数 $0 \le n \le 255$）。这里我们采用行优先（row-major）索引：若原图中某像素坐标为 (x, y)，且

$$0 \le x < w, \qquad 0 \le y < h, \tag{2}$$

则其数组下标为

$$i = i(x, y) = yw + x. \tag{3}$$

这意味着我们按从左到右、从上到下的顺序读取像素。只要知道图像的宽（或高），这个过程就是可逆的。

于是，通过重排全部像素得到的一种混淆方法可定义为一个置换[1]

$$\sigma\colon \{0, 1, \ldots, wh-1\} \to \{0, 1, \ldots, wh-1\} \tag{4}$$

并且混淆后的图像为

$$A' = \left(a_{\sigma(n)}\right)_n. \tag{5}$$

换言之，混淆图像中的第 n 个像素对应于原图中的第 $\sigma(n)$ 个像素。对 A' 应用逆混淆方法 σ^{-1} 可得

$$A'' = \left(a_{\sigma^{-1}(\sigma(n))}\right)_n = \left(a_n\right)_n = A \tag{6}$$

从而恢复原图。

3 混淆算法

在 `obfusimg` 中实现的每种算法本质上都是一个映射

$$\alpha\colon \mathcal{I} \times [0, 1) \to \mathcal{P}, (I, s) \mapsto \sigma \tag{7}$$

其中 $\mathcal{I}$ 是所有图像的集合，$\mathcal{P}$ 是所有有限集合置换的集合，$I = (w, h, c, A)$ 是用户输入图像，$s \in [0, 1)$ 是用户定义的种子（seed），而 σ 是集合 $\{0, 1, \ldots, wh-1\}$ 上的一个置换。实际中，$\alpha(I, s)$ 几乎从不依赖于 c 和 A，因为 c 的变化很小，而对于 `jpeg` 文件来说，A 又容易受到有损压缩影响。

3.1 G/Hilbert 曲线

对于一个 $2^n \times 2^n$ 的正方形，Hilbert 曲线[2]（Gilbert curve）是一条通过相邻像素连续行进、且恰好访问每个像素一次的“连续”曲线。若用曲线经过像素的步数来标记像素索引，则可得到一个置换

$$h\colon \{0, 1, \ldots, 2^{2n}-1\} \to \{0, 1, \ldots, 2^{2n}-1\}, \text{step} \mapsto \text{该步对应的像素索引}. \tag{8}$$

由于 Hilbert 曲线是分形的极限，一个简单递归即可给出任意 n 的 Hilbert 曲线。

对于一个 $w \times h$ 的矩形，广义 Hilbert 曲线（Gilbert 曲线）是 Hilbert 曲线的推广：它同样“连续”且恰好到达每个像素一次（见图 2）。类似地，这条曲线定义了一个置换

$$g\colon \{0, 1, \ldots, wh-1\} \to \{0, 1, \ldots, wh-1\} \tag{9}$$

我们可以将其用作所需的置换 σ。

[1] 映射 $f\colon A \to A$ 若是双射，则称为 A 上的一个置换。因此，置换 $f\colon A \to A$ 必然存在唯一逆映射 $f^{-1}\colon A \to A$。

[2] 严格来说，我们使用的是趋向 Hilbert 曲线的有限次迭代曲线。Hilbert 曲线与 Peano 曲线一样，都是空间填充曲线，并且是从 $[0, 1]$ 到 $[0, 1]^2$ 的连续双射映射的例子。这一点颇为反直觉，因为低维长方体的 C^1 像在 Jordan 测度下为零。

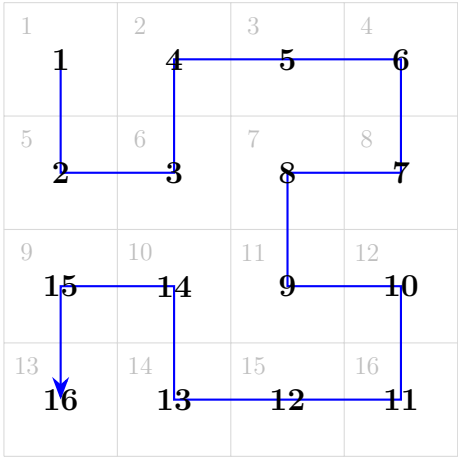


Figure 2: 4×4 矩形上的一个 Gilbert 曲线示例。灰色数字表示原始索引，黑色数字表示重排后的索引。换言之，g(黑色) = 灰色。注意此处的 Gilbert 曲线与 `gilbert_curve.cpp` 中实际实现的 Gilbert 曲线略有不同。

如果朴素地计算 Gilbert 曲线，人们可能会想到：将 $w \times h$ 矩形“填充”到包含它的最小 $2^n \times 2^n$ 正方形，在该正方形上计算 Hilbert 曲线，然后截去矩形之外的曲线部分。然而这样计算效率较低，因为通常会有超过一半的计算资源浪费在矩形外区域。这里我们采用两种更高级的方法。

3.1.1 紧凑 Gilbert 曲线（Compact Gilbert curves）

该方法[3] 由 Chris H. Hamilton 和 Andrew Rau-Chaplin 提出，先计算

$$w^* = \lceil \log_2(w) \rceil, \qquad h^* = \lceil \log_2(h) \rceil \tag{10}$$

它们分别是表示一个 x 索引与 y 索引所需的比特数（0 或 1）。该方法对给定点 (x, y)，从最粗（最高）比特层开始，判断该点在当前比特层所处象限并设置对应比特，然后进入下一层（更低层），重复此过程。

在过程结束时，可以得到一个索引 $0 \le C(x, y) \le 2^{w^*+h^*} - 1$，它描述了该点在 Gilbert 曲线上的位置。对所有 $0 \le x < w$ 与 $0 \le y < h$ 计算 $C(x, y)$ 后，得到未归一化紧凑 Gilbert 函数

$$C\colon \{0, 1, \ldots, wh - 1\} \to \{0, 1, \ldots, 2^{w^*+h^*} - 1\}. \tag{11}$$

该映射是单射，但通常不是满射。我们通过对输出值排序并取秩来归一化 C，得到归一化紧凑 Gilbert 函数

$$\tilde{g}\colon N = \{0, 1, \ldots, wh - 1\} \to N, i \mapsto C(i) \text{ 在 } C(N) \text{ 中的秩}. \tag{12}$$

这个 $\tilde{g}$ 满足：$\tilde{g}(i)$ 是索引为 i 的点在“截断曲线”上的位置。对 $\tilde{g}$ 取逆可得

$$g\colon \{0, 1, \ldots, wh - 1\} \to \{0, 1, \ldots, wh - 1\}, i \mapsto \tilde{g}^{-1}(i) \tag{13}$$

它给出了曲线上第 i 个点对应的像素索引。这个 g 就是我们所需的置换 σ。

[3]参见 “Compact Hilbert indices: Space-filling curves for domains with unequal side lengths.” 可见 https://doi.org/10.1016/j.ipl.2007.08.034.

3.1.2 精确 Gilbert 曲线（Exact Gilbert curves）

该方法[4] 由 Jakub Červený 提出，通过递归地将矩形沿 U 形路径分成三个区域，构造一条完全在 $w \times h$ 矩形内部行进的曲线。注意在某些特殊情况下会使用对角移动。该方法给出一个归一化精确 Gilbert 函数

$$g\colon \{0, 1, \ldots, wh-1\} \to \{0, 1, \ldots, wh-1\} \tag{14}$$

它给出了曲线上第 i 个点对应的像素索引。

参考常用工具“小番茄图片混淆”[5]，在这一情形下我们并不直接将此 g 用作置换。相反，我们沿精确 Gilbert 曲线将每个像素整体平移一个偏移量

$$k = \left\lfloor \frac{\sqrt{5}-1}{2} hw \right\rceil = \left\lfloor \frac{hw}{\varphi} \right\rceil. \tag{15}$$

这意味着：新图像中索引为 j 的像素（它原本是曲线上第 $g^{-1}(j)$ 个点）在平移后应当来自曲线上第 $(g^{-1}(j) - k)$ 个点。由于 g 为双射，存在某个 i 使得 $j = g(i)$。因此我们所需的置换为

$$\sigma\colon \{0, 1, \ldots, wh-1\} \to \{0, 1, \ldots, wh-1\}, g(i) \mapsto g(\overline{i-k}) \tag{16}$$

其中 $\overline{i-k}$ 按 mod hw 理解。注意由于 g 是双射，该置换是良定义的。

3.2 混沌系统

如果我们只想得到一个高度“噪声化”但可复现的 σ（例如 Cloudflare 旧金山办公室的熔岩灯能够产生非常嘈杂的数据，但之后几乎不可能反求出置换），那么可以考虑混沌系统。非常宽松地说[6]，我们用以下性质来刻画混沌系统：

- 系统以确定性方式演化。也就是说，总存在明确函数 f，使得 $x_{n+1} = f(x_n)$。
- 对任意初始状态，只要时间足够长，系统都可以演化到任意区域。
- 系统对初始条件敏感。即两个非常接近的初始点，之后可能演化出截然不同的轨迹。

有许多简单函数 $f\colon I \subseteq \mathbb{R} \to \mathbb{R}$ 会表现出混沌行为。例如，当 $r \lesssim 4$ 时的 logistic 映射 $L(x) = rx(1-x)$，参数 $\mu \in (1, 2)$ 时的 tent 映射 $T(x) = \mu \min\{x, 1-x\}$（见图 3），以及倍增映射 $D(x) = 2x \pmod 1$，都可用于模拟混沌。

设一个混沌映射 $f\colon I \subseteq \mathbb{R} \to \mathbb{R}$。用户定义的种子 s 给定初始条件 $x_0 = s$。对 $0 \le i < wh-1$，计算

$$x_{i+1} = f(x_i). \tag{17}$$

这将得到一个类似噪声的数组 $(x_n)_n$。我们可通过排序对 x_n 取秩，并在必要时使用下标 n 打破并列，从而得到所需置换

$$\sigma\colon \{0, 1, \ldots, wh-1\} \to \{0, 1, \ldots, wh-1\}, i \mapsto \text{使得 } x_n \text{ 的秩为 } i \text{ 的那个 } n. \tag{18}$$

[4]参见 https://github.com/jakubcerveny/gilbert/.

[5]参见 https://xfqtphx.netlify.app/.

[6]严格地讲，混沌系统有多种严谨数学定义。最经典的一种是定义在度量空间 (X, d) 上连续函数 $f\colon X \to X$ 的 Devaney 混沌：若 f 拓扑传递、周期点稠密，并且对初值敏感，则称其为 Devaney 混沌。不过这里我们把这些内容留到数学课中讨论。

3.2.1 Tent 映射

我们实现的 tent 映射为

$$T_\mu\colon [0,1] \to [0,1], x \mapsto \begin{cases} \mu x & x < 1/2, \\ \mu(1-x) & x \geq 1/2 \end{cases} \tag{19}$$

其中 $0 < \mu \leq 2$。该映射在 $\mu = 2$ 时具有很强的混沌性。然而在实际 C++ 实现中，乘以 2 基本等价于按位左移。由于精度有限，大约迭代 53 次后所有有效比特都会丢失。因此我们取 $\mu = 1.9999$。

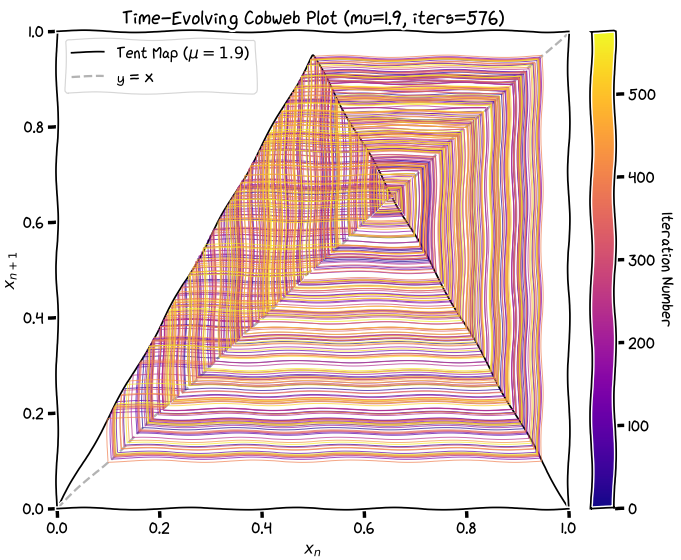


Figure 3: 取 $\mu = 1.9$、$x_0 = 0.7$ 时，tent 映射迭代 24^2 次后的演化。

附录

A permutations 模块的头文件

```cpp
/// @file permutation.h
/// @author __roselle__, pigeon.hannah
/// @date 2026-02-20
/// @brief A module that handles permutations encoded as vectors of integers.
#include <vector>

/// @brief Function to normalize a given permutation by associating each
///        element with its rank in the permutation.
/// @param perm  the permutation to be normalized (std::vector<int>)
/// @return the normalized permutation (std::vector<int>)
std::vector<int> normalize_permutation(const std::vector<int>& perm);

/// @brief Function that computes a permutation by shifting a sequence a(n)
///        such that a(n) lands in the position of a(n + shift).
/// @param seq   the original sequence to be shifted (std::vector<int>)
/// @param shift the number of positions to shift the sequence (int)
/// @return the shifted sequence as a permutation (std::vector<int>)
std::vector<int> shift_sequence(const std::vector<int>& seq, int shift);

/// @brief Function to invert a given permutation.
/// @param perm  the permutation to be inverted (std::vector<int>)
/// @return the inverted permutation (std::vector<int>)
std::vector<int> invert_permutation(const std::vector<int>& perm);

/// @brief Function to apply a permutation to an image's pixel data.
/// @param pixels    the pixel data of the original image (unsigned char*)
/// @param perm      the permutation to be applied to the pixel data
///                  (std::vector<int>)
/// @param width     the width of the image (int)
/// @param height    the height of the image (int)
/// @param channels  the number of channels in the image (int)
/// @return the obfuscated pixel data (unsigned char*)
unsigned char* apply_permutation(
    unsigned char* pixels, std::vector<int> perm,
    int width, int height, int channels
);
```

B Gilbert 曲线模块的头文件

```
/// @file gilbert_curve.h
/// @author __roselle__, pigeon.hannah
/// @date 2026-02-20
/// @brief A module that computes a generalized hilbert curve for an arbitrary
///        2-dimensional rectangle.
#include <vector>

/// @brief Function to compute the nth bit of an integer in binary.
/// @param val the interger to be represented in binary (int)
/// @param bit the position of the bit to be extracted,
///        where 0 is the least significant bit (int)
/// @return the value of the nth bit (0 or 1) (int)
int get_bit(int val, int bit);

/// @brief Function to compute the sign of an integer.
/// @param x  the integer whose sign is to be computed (int)
/// @return 1 if x is positive, -1 if x is negative, and 0 if x is zero (int)
int sign(int x);

/// @brief Function to perform floor division by 2 for integers.
/// @param x  the integer to be divided (int)
/// @return the result of floor division of x by 2 (int)
int floor_div2(int x);

/// @brief Function to compute the Compact Hilbert Index for a 2D point.
///        The index can be imagined as the position of the point along a
///        Gilbert curve that fills the smallest containing rectangle of size
///        2^w_prec by 2^h_prec.
/// @param x     the horizontal coordinate of the point (int)
/// @param y     the vertical coordinate of the point (int)
/// @param w_prec the precision (number of bits) needed to represent the
///               width of the rectangle (int)
/// @param h_prec the precision (number of bits) needed to represent the
///               height of the rectangle (int)
/// @return the Compact Hilbert Index corresponding to the point (int)
/// @cite https://doi.org/10.1016/j.ipl.2007.08.034
int get_compact_index(int x, int y, int w_prec, int h_prec);

/// @brief Function to compute the compact g function for a given rectangle.
///        This g function needs to be normalized and inverted.
/// @param w  the width of the rectangle (int)
/// @param h  the height of the rectangle (int)
/// @return a vector of integers representing the $g$ function, where the value
///        at index $i$ is the Compact Hilbert Index of the point corresponding
///        to the original index $i$ in row-major order (std::vector<int>)
std::vector<int> generate_compact_g_function(int w, int h);

/// @brief Recursive function to compute the coordinates of the points along
```

```
    an
///         exact Gilbert curve by dividing the rectangle into smaller blocks.
/// @param x          the starting absolute horizontal coordinate of the
///                   current block (int)
/// @param y          the starting absolute vertical coordinate of the
///                   current block (int)
/// @param ax         the width of the major axis of the current block with
///                   signature indicating movement direction (int)
/// @param ay         the height of the major axis of the current block with
///                   signature indicating movement direction (int)
/// @param bx         the width of the minor axis of current block with
///                   signature indicating movement direction (int)
/// @param by         the height of the minor axis of current block with
///                   signature indicating movement direction (int)
/// @param img_width the total width of the image, used to compute 1D indices
///                   from 2D coordinates (int)
/// @param coordinates reference to the output vector (std::vector<int>)
/// @cite https://github.com/jakubcerveny/gilbert/
void draw_gilbert_curve(
    int x, int y, int ax, int ay, int bx, int by, int img_width,
    std::vector<int>& coordinates
);

/// @brief Function to compute the exact g function for a given rectangle.
///        This g function is normalized.
/// @param w  the width of the rectangle (int)
/// @param h  the height of the rectangle (int)
/// @return a vector of integers representing the $g$ function, where the
    value
///        at index $i$ is the row-major index of the $i$-th point in the exact
///        Gilbert curve (std::vector<int>)
std::vector<int> generate_exact_g_function(int width, int height);
```

C 混沌系统模块的头文件

```
/// @file chaos.h
/// @author __roselle__, pigeon.hannah
/// @date 2026-02-20
/// @brief A module that computes a chaotic permutation based on various
///        chaotic maps and an initial seed.
#include <vector>
#include <functional>

/// @brief Computes the value of the tent map at a given point.
/// @param x  the input value, typically in the range [0, 1] (double)
/// @param mu the parameter of the tent map,
///           note that mu = 2.0 loses chaos after about 53 iterations due to
///           binary representation and should be avoided (double)
/// @return the output value of the tent map (double)
double tent_map(double x, double mu);

/// @brief Generates a chaotic permutation of integers from 0 to n-1 based on
///        the a chaotic map and an initial seed.
/// @param n     the size of the permutation (int)
/// @param seed  the initial seed for the chaotic map (double)
/// @param map_func the chaotic map to be used (std::function<double(double)>)
/// @return a vector of integers representing the chaotic permutation, where
std::vector<int> generate_chaotic_permutation(
    int n, double seed, std::function<double(double)> map
);
```